

High-Level Design

1. System Overview

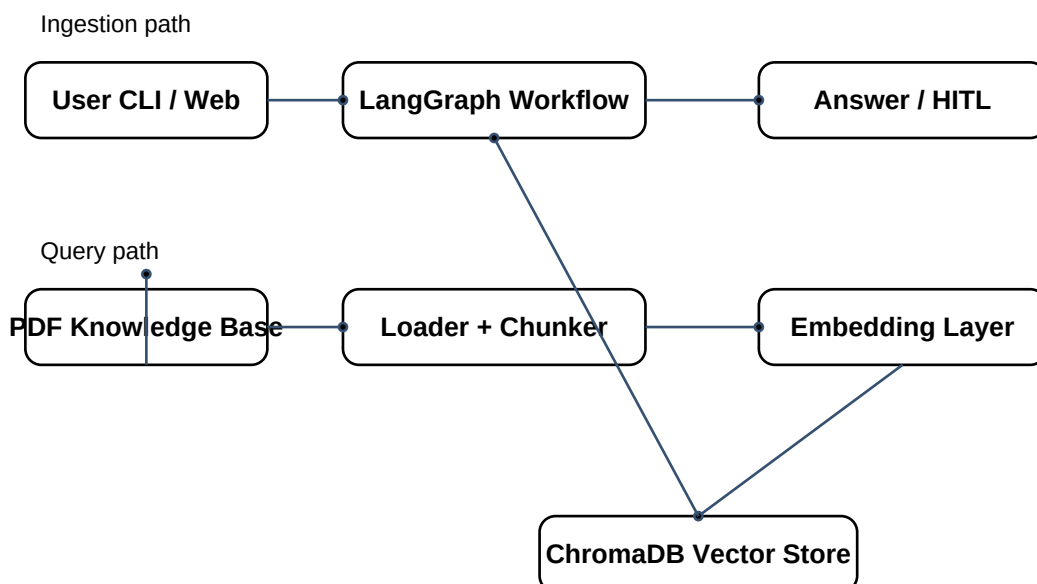
The system solves a common customer-support problem: support agents and end users need quick, context-aware answers from a policy document, but manual document lookup is slow and inconsistent. A Retrieval-Augmented Generation (RAG) assistant improves response quality by grounding answers in a trusted PDF knowledge base before generating a response.

The scope of the system includes:

- ingesting a PDF knowledge base
- extracting and chunking document text
- generating embeddings for chunks
- storing vectors in ChromaDB
- retrieving relevant context at query time
- orchestrating answer generation with LangGraph
- routing low-confidence or sensitive queries to a Human-in-the-Loop (HITL) path

The system is intentionally designed as a customer-support assistant, but the architecture is reusable for internal help desks, policy bots, and operations playbooks.

2. Architecture Diagram



3. Component Description

User Interface

The reference implementation uses a CLI. The same service layer can later support a web or chat UI because the workflow is packaged as a Python service object.

Document Loader

The document loader reads the PDF knowledge base and extracts page-level text using ``pypdf``. Page-level metadata is preserved so final answers can cite the original source.

Chunking Strategy

The chunker uses fixed-size character windows with overlap. This keeps retrieval units small enough for precise matching while still preserving continuity across chunk boundaries.

Embedding Model

The implementation uses a deterministic local hashing embedder so the project remains runnable offline. The design keeps the embedder behind an interface, allowing replacement with a production embedding model without changing the rest of the pipeline.

Vector Store

ChromaDB stores chunk text, metadata, and vector embeddings. It provides persistent local storage, simple collection management, and fast top-k similarity search for a self-contained project.

Retriever

The retriever queries ChromaDB using the embedded user query and returns the most relevant chunks with similarity-derived confidence signals.

LLM Processing Layer

The answer generator composes a grounded response from retrieved chunks. In the demo build this is an offline synthesis layer. In production, this layer can be swapped with a hosted chat model while preserving the same workflow contract.

Graph Workflow Engine

LangGraph coordinates the end-to-end control flow. The core flow is ``Input -> Process -> Output``, extended with conditional routing to a HITL branch when the query is high-risk or low-confidence.

Routing Layer

The routing layer detects intent, assesses retrieval confidence, and decides whether the assistant should answer directly or escalate.

HITL Module

The HITL module creates escalation tickets, records reasons, and supports later insertion of a human-approved response.

4. Data Flow

PDF to Answer Flow

1. A PDF knowledge base is loaded from disk.
2. Text is extracted page by page.
3. The text is chunked into overlapping segments.
4. Each chunk is embedded and stored in ChromaDB with metadata.
5. A user submits a support question.
6. The query is embedded and matched against stored chunks.
7. The workflow evaluates the query intent and retrieval quality.
8. The assistant either generates a grounded answer or creates a HITL escalation ticket.
9. The final response is returned with citations or escalation details.

Query Lifecycle

The query lifecycle begins at the interface boundary, passes through intent detection and retrieval, then reaches a branching decision. The normal branch produces an answer from retrieved context. The exception branch queues the case for human review and returns a controlled escalation message.

5. Technology Choices

Why ChromaDB

- lightweight local deployment
- persistent vector storage without external infrastructure
- simple Python API that suits project demonstrations and prototypes
- easy upgrade path to hosted vector infrastructure later

Why LangGraph

- explicit graph-based control flow
- clean support for conditional routing
- easy extension from a linear flow to a richer state machine
- well-suited for agentic and HITL-oriented orchestration

LLM Choice

The design treats generation as an adapter layer. The included implementation uses an offline response synthesizer so the project can run without credentials. A production deployment can replace that adapter with a managed chat model and keep the same retrieval and workflow contracts.

Additional Tools

- ``pypdf`` for PDF extraction
- ``reportlab`` for generating final PDF deliverables
- Python ``argparse`` for the CLI interface

6. Scalability Considerations

Handling Large Documents

Large PDFs can be processed in batches and indexed incrementally. Chunk metadata can include document IDs, section IDs, and versioning for multi-document retrieval.

Increasing Query Load

For higher throughput, the retrieval and generation layers can be wrapped in an API service and horizontally scaled. ChromaDB can be replaced with a higher-capacity vector database if concurrent load increases significantly.

Latency Concerns

The main latency contributors are embedding, retrieval, and answer generation. Pre-computed document embeddings already reduce ingestion-time cost at query time. Retrieval depth, chunk size, and output prompt length should be tuned together to keep latency acceptable.